PINN_M3DC1

Module 4

Data-Only Parameterized Surrogate Baseline

A textbook-style introduction to supervised field learning, fair baselines, and reduced-MHD generalization

Learning task

Learn the continuous parameterized map

$$(x, y, t, \eta, \nu, A_0) \longmapsto (\widehat{\psi}, \widehat{U})$$

from verified numerical field values alone, without using a partial differential equation, initial-condition loss, boundary-condition loss, or global-balance loss during training.

Version 1.0 | 4 August 2026

Physics and scientific-machine-learning note for the synthetic reduced-MHD proof of concept

Preface: why a data-only model belongs in a PINN project

This project is ultimately interested in a *physics-informed neural network*, abbreviated PINN. It may therefore seem natural to begin directly with the physics-informed model in Module 5. That would be a scientific mistake. If a PINN predicts the fields accurately, the result by itself does not tell us *why*. The network may have learned almost everything from the numerical observations, while the partial differential equation (PDE) constraints contributed little. Alternatively, the physics constraints may substantially improve accuracy, data efficiency, extrapolation, or physical consistency. A controlled data-only baseline is what distinguishes those possibilities.

Module 4 therefore asks a deliberately restricted question:

> Given numerical solutions of the two-field reduced resistive-magnetohydrodynamic model, how accurately can an ordinary supervised neural network learn the solution fields at new points and at entirely unseen physical parameter combinations?

The initials **MHD** mean *magnetohydrodynamics*, the fluid description of an electrically conducting medium coupled to a magnetic field. The word **surrogate** means a computational model designed to approximate the input-output behavior of a more expensive reference calculation. The term **data-only** means that the training objective compares predictions with stored numerical values but contains no equation residual or other physical constraint. The term **parameterized** means that physical case parameters are explicit inputs, allowing one network to represent a family of simulations rather than only one trajectory.

This note assumes no previous course in machine learning. It begins with the meaning of supervised learning and builds the complete baseline experiment one object at a time. Familiarity with multivariable calculus, matrices, and the Module 1 definitions of magnetic flux ψ, stream function U, current J, and vorticity ω is helpful. All learning-specific notation is defined before it is used.

Important distinction

The Module 4 network is *not* a PINN. It is a conventional supervised coordinate network. Reduced-MHD equations may be used after training to diagnose its predictions, but they may not enter its training loss. If a PDE residual, initial-condition penalty, periodic-boundary penalty, or energy-balance penalty affects parameter updates, the experiment has moved into Module 5.

Important distinction : claim boundary

The initial project data are generated by an independent two-dimensional reduced-MHD solver. Consequently, Module 4 establishes a data-only surrogate of that synthetic model, not a validated surrogate of genuine M3D-C1 output. Actual M3D-C1 arrays and renewed validation are required for the stronger claim reserved for Module 7.

How to read this note

The main conceptual chain is

$$\text{verified simulation cases} \longrightarrow \text{paired inputs and targets} \longrightarrow \text{case-wise partitions}$$
$$\longrightarrow \text{scaling and periodic encoding} \longrightarrow \text{neural network}$$
$$\longrightarrow \text{data loss and optimization} \longrightarrow \text{unseen-case evaluation.}$$

Sections 1–3 define the scientific question and the exact learned map. Sections 4–7 construct a defensible dataset and input representation. Sections 8–11 explain the neural network and its training. Sections 14–18 define physical evaluation and the controlled comparison with Module 5. Sections 19–24 freeze the recommended baseline and implementation handoff.

Learning objectives

After working through this module, the reader should be able to:

1. distinguish the reference solver, training dataset, neural surrogate, and physics-informed surrogate;
2. explain why a data-only baseline is required to measure the value of physics information;
3. write the project map $(x, y, t, \eta, \nu, A_0) \mapsto (\widehat{\psi}, \widehat{U})$ and define every symbol;
4. distinguish a pointwise coordinate network from a time-stepper and a neural operator;
5. convert gridded simulation cases into supervised examples without losing case identity;
6. prevent leakage by splitting complete parameter cases before selecting points or computing scaling statistics;
7. distinguish physical nondimensionalization from numerical feature standardization;
8. encode periodic coordinates so that $x = 0$ and $x = L_x$ are represented consistently;
9. derive the forward equations of a multilayer perceptron and count its trainable parameters;
10. explain activation functions, backpropagation, mini-batches, epochs, and the Adam optimizer;
11. construct a channel-balanced, case-balanced data loss for ψ and U;
12. separate model fitting, hyperparameter selection, and final testing;
13. distinguish spatial interpolation, temporal interpolation, parameter interpolation, and parameter extrapolation;

14. evaluate J, ω, energies, periodicity, and PDE residuals without using them to train Module 4;
15. design a fair data-budget comparison between the data-only baseline and Module 5 PINN; and
16. identify common failure modes such as random-point leakage, output-scale imbalance, spectral bias, gauge drift, and misleading contour plots.

Contents

1 Scientific role of Module 4

1.1 Four different computational objects

Several objects in this project can all produce field values, but they are not interchangeable.

Object	What it does	What determines its output
Reduced-MHD reference solver	Numerically integrates the frozen two-field PDEs from a specified initial condition.	PDEs, initial and boundary conditions, coefficients, grid, time step, and numerical method.
Stored dataset	Records selected solver coordinates, parameters, fields, metadata, and diagnostics.	Reference runs and the versioned data contract from Module 3.
Module 4 data-only surrogate	Fits stored field values by supervised learning.	Network architecture, observed data, scaling, sampling, loss, optimizer, and training choices.
Module 5 physics-informed surrogate	Fits data while also penalizing violations of the governing physics.	Everything used in Module 4 plus PDE, initial-condition, boundary-condition, and optional global constraints.

The reference solver supplies the best available numerical approximation to the selected reduced model. The word *reference* does not imply exact truth. Its discretization errors must be quantified in Module 2. The neural network learns the stored numerical solutions, so it cannot be more faithful to the mathematical PDE than the data source merely by reproducing the source closely.

1.2 The controlled scientific question

Let M_{data} denote the trained data-only model and M_{phys} the later physics-informed model. The central comparison is not

"Is M_{phys} accurate?"

but rather: *At the same data budget and on the same unseen cases, what changes when physical constraints are added?* Possible measured outcomes include:

- lower or higher field error;
- lower or higher derived-current and vorticity error;
- reduced or unchanged training-data requirement;

- better or worse parameter generalization;
- improved or degraded satisfaction of periodicity and global balances;
- different training cost, memory use, or optimization reliability; and
- different sensitivity to random initialization.

A negative result is scientifically meaningful. Physics information is not guaranteed to help: the residual may be badly scaled, the model may be difficult to optimize, or the data may already constrain the relevant solution family sufficiently.

1.3 What "baseline" means

Definition

A **baseline** is a clearly specified comparison method that answers the same prediction question under controlled conditions. A useful baseline is not intentionally weak. It should be competent enough that any improvement by a new method cannot be attributed to an avoidable defect in the comparator.

For this project, a fair baseline should have adequate network capacity, appropriate scaling, a periodic input representation, stable optimization, and careful validation. Removing physics losses does not justify using poor architecture or poor data handling.

Learning checkpoint

If Module 5 outperforms a badly trained Module 4 network, the result demonstrates only that one training setup was better than another. If the models share architecture, observations, preprocessing, case splits, and evaluation, then their difference can be attributed much more directly to the added physics terms and their optimization consequences.

2 From a simulation family to supervised learning

2.1 What supervised learning means

In *supervised learning*, the available data contain both an input and a desired output. An individual input is often called a *feature vector*; the desired output is called a *target*, *label*, or *response*. The network evaluates a parameterized function

$$\widehat{\boldsymbol{q}} = \mathcal{N}_{\boldsymbol{\theta}}(\boldsymbol{z}), \tag{2.1}$$

where

- $\boldsymbol{z}$ is the input feature vector;
- $\widehat{\boldsymbol{q}}$ is the predicted output vector;
- $\mathcal{N}$ denotes the neural-network computation; and
- $\boldsymbol{\theta}$ collects every trainable weight and bias.

The hat over $\widehat{\boldsymbol{q}}$ means "predicted" or "approximated." The corresponding reference value without a hat is $\boldsymbol{q}$.

Training adjusts $\boldsymbol{\theta}$ to reduce a numerical measure of disagreement between $\widehat{\boldsymbol{q}}$ and $\boldsymbol{q}$. That measure is the *loss function*. In Module 4, the disagreement is evaluated only against stored data.

2.2 One case versus a family of cases

A single reduced-MHD simulation has fixed values of the case parameters

$$\boldsymbol{\mu} = (\eta, \nu, A_0), \tag{2.2}$$

where η is the dimensionless magnetic diffusivity, ν is the dimensionless kinematic viscosity, and A_0 is the dimensionless initial perturbation amplitude. The solver produces fields over space and time:

$$\psi = \psi(x, y, t; \boldsymbol{\mu}), \qquad U = U(x, y, t; \boldsymbol{\mu}). \tag{2.3}$$

The semicolon is a visual separator: x, y, and t locate a point within one solution, whereas $\boldsymbol{\mu}$ selects which physical case is being considered.

A separate network could be trained for every $\boldsymbol{\mu}$. Such a collection would not be a parameterized surrogate and could not predict a new coefficient combination without retraining. Instead, the project gives $\boldsymbol{\mu}$ to the network as part of its input. One set of parameters $\boldsymbol{\theta}$ then attempts to represent an entire finite-dimensional solution family.

2.3 Approximation, interpolation, and generalization

The word *approximation* means that the neural output is close to a target function according to a stated metric. *Interpolation* means prediction within a region supported by training inputs. *Extrapolation* means prediction outside that support. *Generalization* is the broader ability to perform well on examples not used to update the model.

These words must be qualified because the input has six dimensions. A point may interpolate in x and y, extrapolate in time, and interpolate in (η, ν, A_0) simultaneously. The project therefore reports which axes are held out rather than using "generalization" without qualification.

3 The exact Module 4 surrogate map

3.1 Primary input and output variables

The unprocessed physical input is

$$\boldsymbol{s} = \begin{bmatrix} x & y & t & \eta & \nu & A_0 \end{bmatrix}^{\mathsf{T}}, \tag{3.1}$$

where superscript T denotes transpose, turning a row into a column. The primary target is

$$\boldsymbol{q} = \begin{bmatrix} \psi & U \end{bmatrix}^{\mathsf{T}}. \tag{3.2}$$

The intended learned function is

$$\boxed{\mathcal{N}_{\boldsymbol{\theta}} : (x, y, t, \eta, \nu, A_0) \longmapsto (\widehat{\psi}, \widehat{U}).} \tag{3.3}$$

All six inputs and both outputs are dimensionless under the physical normalization frozen in Module 1.

The word *continuous* here means that, once trained, the network can be evaluated at coordinates between stored grid points. It does not mean those off-grid predictions are guaranteed accurate. Their accuracy must be tested against refined or independently interpolated reference solutions.

3.2 Why J and ω are not primary targets

The reduced-MHD definitions are

$$J = -\nabla^2 \psi, \qquad \omega = \nabla^2 U, \tag{3.4}$$

where

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \tag{3.5}$$

is the two-dimensional Laplacian. J is the project current variable and ω is the out-of-plane vorticity.

The primary baseline predicts ψ and U because these are the evolved potentials and because Module 5 will require differentiable versions of them. Current and vorticity are then derived from the predicted fields. This choice tests whether the network captures spatial curvature, not merely whether it can regress four separately supplied arrays.

Important distinction

Small pointwise error in ψ or U does not guarantee small error in J or ω. Second derivatives amplify short-wavelength error. A smooth contour plot may hide oscillations whose amplitude is small but whose curvature is large.

An optional four-output network $(\widehat{\psi}, \widehat{U}, \widehat{J}, \widehat{\omega})$ would answer a different question. It may be useful as an ablation, but it should not silently replace the frozen primary baseline. If used, consistency defects

$$\widehat{J} + \nabla^2 \widehat{\psi}, \qquad \widehat{\omega} - \nabla^2 \widehat{U} \tag{3.6}$$

must be reported. Penalizing those defects during training would already add physics or differential consistency and would therefore no longer be the strict value-only baseline.

3.3 Coordinate network, time-stepper, and neural operator

Three learning formulations are often conflated.

Formulation	Typical map	Interpretation
Coordinate network	$(x, y, t, \boldsymbol{\mu}) \mapsto \boldsymbol{q}$	Returns the field value at a requested location, time, and parameter setting.
Time-stepper	$\boldsymbol{q}(t_n) \mapsto \boldsymbol{q}(t_{n+1})$	Advances an entire discrete state from one stored time to the next, often autoregressively.
Neural operator	input function $\mapsto$ output function	Learns a mapping between function spaces, for example an arbitrary initial field to a later solution field.

Module 4 uses a **coordinate network**. It is not an autoregressive time integrator: evaluation at time t does not require the network prediction at an earlier time. It is also not a general neural operator because the initial condition is represented only through the finite parameter A_0, not supplied as an arbitrary function. Operator-learning architectures such as DeepONet or a Fourier neural operator may become relevant when the input includes general profiles, equilibria, or geometries [7, 8]; they are outside the frozen first baseline.

3.4 What the network is really approximating

If the numerical solver and data pipeline were exact and deterministic, there would be a solution map

$$\mathcal{S} : (x, y, t, \boldsymbol{\mu}) \mapsto (\psi, U). \tag{3.7}$$

The trained network seeks

$$\mathcal{N}_{\boldsymbol{\theta}} \approx \mathcal{S} \tag{3.8}$$

over a declared domain. In practice the targets also contain discretization and interpolation errors. It is more precise to say that the network approximates the *verified discrete reference solution family* over the sampled parameter and time domain.

4 Anatomy of the simulation dataset

4.1 Case, snapshot, grid point, and example

The following hierarchy prevents ambiguous use of the word "sample."

Definition

A **case** is one complete simulation with a fixed parameter vector $\boldsymbol{\mu}_c = (\eta_c, \nu_c, A_{0,c})$ and fixed numerical metadata. A **snapshot** is the complete spatial field at one stored time t_k. A **grid point** is one spatial coordinate pair (x_i, y_j). A **supervised example** pairs one network input with its target output.

For case c, suppose the data contain N_x x coordinates, N_y y coordinates, and N_t stored times. The arrays may be represented as

$$\psi_{ijk}^{(c)} = \psi(x_i, y_j, t_k; \boldsymbol{\mu}_c), \qquad U_{ijk}^{(c)} = U(x_i, y_j, t_k; \boldsymbol{\mu}_c). \tag{4.1}$$

If every stored point is used, one case contains

$$N_{\text{point}} = N_x N_y N_t \tag{4.2}$$

supervised examples. With N_{case} cases, the full point count is the sum over cases. This count can be very large even when the number of independent physical trajectories is modest.

4.2 Flattening arrays without erasing structure

To train a pointwise network, a selected array entry becomes

$$\boldsymbol{s}^{(c,i,j,k)} = (x_i, y_j, t_k, \eta_c, \nu_c, A_{0,c}), \tag{4.3}$$

$$\boldsymbol{q}^{(c,i,j,k)} = (\psi_{ijk}^{(c)}, U_{ijk}^{(c)}). \tag{4.4}$$

These examples may be stored as rows of an input matrix and target matrix. Flattening is a storage and batching operation; it must not discard the case identifier, snapshot index, original coordinate indices, split assignment, or provenance. Those fields are needed for case-balanced losses and reconstruction of full evaluation grids.

4.3 Why point count is not independent-information count

Adjacent grid points within one smooth field are strongly correlated. Neighboring stored times are also correlated because they lie on the same trajectory. Therefore, 10^7 point examples from ten cases do not provide the same diversity as 10^7 examples from thousands of independent physical cases. Reporting only the number of points can exaggerate dataset breadth.

A complete data report should include at least:

- number of cases in each split;
- the parameter values or parameter ranges in each split;
- spatial resolution and stored-time resolution;
- number of point observations actually used for training;
- sampling strategy within cases; and
- whether repeated points are resampled between epochs.

4.4 Required metadata inherited from Module 3

Each case must preserve domain lengths (L_x, L_y), coordinate arrays, time array, parameter vector, normalization, sign convention, initial and boundary conditions, solver version, discretization, diagnostic definitions, and split assignment. The training pipeline should reject a case with incompatible conventions rather than silently concatenate it.

Important distinction

Plot images are not field data. An image loses numerical precision, coordinates, normalization, field names, and usually the relationship between variables. It is useful for human inspection but is not an acceptable regression target.

5 Partitioning data and defining generalization

5.1 Training, validation, and test sets

The complete collection of cases is divided into three disjoint subsets.

- The **training set** updates the neural parameters $\boldsymbol{\theta}$.
- The **validation set** selects hyperparameters, training duration, architecture, and checkpoints.
- The **test set** is used once the design is frozen to estimate final performance on unseen cases.

A *hyperparameter* is a choice made outside ordinary gradient updates, such as width, depth, learning rate, weight decay, batch size, activation, or Fourier-feature scale. Choosing the best model after repeatedly inspecting test performance converts the test set into a validation set and produces an optimistically biased result.

5.2 The split unit must be a complete physical case

Let $\mathcal{C}_{\mathrm{tr}}$, $\mathcal{C}_{\mathrm{va}}$, and $\mathcal{C}_{\mathrm{te}}$ be disjoint sets of case identifiers:

$$\mathcal{C}_{\mathrm{tr}} \cap \mathcal{C}_{\mathrm{va}} = \varnothing, \quad \mathcal{C}_{\mathrm{tr}} \cap \mathcal{C}_{\mathrm{te}} = \varnothing, \quad \mathcal{C}_{\mathrm{va}} \cap \mathcal{C}_{\mathrm{te}} = \varnothing. \tag{5.1}$$

All points and all stored times from a given parameter combination must remain in one partition for the principal parameter-generalization experiment.

Important distinction : random-point leakage

Randomly assigning points from every trajectory to training and test sets is not a valid test of unseen-case generalization. The network sees the same (η, ν, A_0) combination and neighboring points from the same numerical trajectory during training. It is then tested mainly on interpolation within an already observed solution.

The correct ordering is:

1. enumerate and validate complete cases;
2. assign whole cases to train, validation, and test partitions;
3. freeze the split manifest;
4. compute preprocessing statistics from training cases only; and
5. sample point observations within each already assigned partition.

5.3 Parameter interpolation and extrapolation

Suppose training covers $\eta \in [\eta_{\min}, \eta_{\max}]$. A held-out η strictly within this interval is an interpolation test in that coordinate, although the complete triplet $\boldsymbol{\mu}$ is unseen. A value outside the interval is an extrapolation test. In three parameter dimensions, the convex hull of the training parameter vectors gives a more careful geometric distinction: a test point inside that hull is supported by surrounding training cases, while a point outside requires extrapolation in at least one combined direction.

The project should maintain separate interpolation and extrapolation test manifests. Averaging them into one score hides a physically important difference.

5.4 Temporal interpolation is not forecasting

If all training cases provide data over the full interval $0 \le t \le T$, and a test case also lies in that time interval, the model is interpolating in the represented time domain even though the parameter combination is unseen. This is not evidence of forecasting beyond T.

A temporal-extrapolation experiment must deliberately train only on $0 \le t \le T_{\text{train}}$ and evaluate at $t > T_{\text{train}}$. This is difficult for a direct coordinate network and must be reported separately. A model can succeed in parameter interpolation yet fail in time extrapolation.

5.5 Spatial interpolation and resolution transfer

Evaluation at coordinates not present in the training point sample tests spatial interpolation. Evaluation on a denser grid is sometimes called super-resolution, but a coordinate network's ability to return values at more coordinates does not by itself prove more physical information has been recovered. The predictions must be compared with a higher-resolution reference calculation, and derivative-sensitive quantities must be checked.

6 Physical nondimensionalization and neural preprocessing

6.1 Two different transformations

Module 1 converts dimensional variables to the dimensionless reduced-MHD system using physical reference scales such as L_0, B_0, and the Alfvén time. Module 4 may then transform the already dimensionless numbers to ranges that are easier for neural optimization.

Transformation	Purpose	Consequence
Physical nondimensionalization	Express the governing physics relative to meaningful reference scales.	Determines the dimensionless PDE coefficients and physical interpretation.
Neural scaling or standardization	Improve numerical conditioning of learning inputs and outputs.	Changes the coordinates seen by the network; must be invertible and fitted only on training data.

Important distinction

Neural scaling does not change the physical model. Conversely, setting every machine-learning input to order one does not automatically nondimensionalize the PDE. Module 5 derivatives must include the chain-rule factors associated with both transformations.

6.2 Affine standardization

For a scalar variable a, training-set standardization is

$$\widetilde{a} = \frac{a - m_a}{s_a}, \tag{6.1}$$

where m_a is a location statistic, commonly the training mean, and $s_a > 0$ is a scale statistic, commonly the training standard deviation. The tilde denotes a numerically transformed variable. The inverse is

$$a = m_a + s_a \widetilde{a}. \tag{6.2}$$

For deterministic coordinates with known bounds, an affine map to $[-1, 1]$ is also convenient:

$$\widetilde{t} = 2\frac{t - t_{\min}}{t_{\max} - t_{\min}} - 1. \tag{6.3}$$

The statistics must be stored with the model. A predicted standardized output is not a physical field until the inverse transform has been applied.

6.3 Scaling positive transport coefficients

If η and ν span several orders of magnitude and are sampled approximately logarithmically, it is often more natural to transform

$$r_\eta = \log_{10} \eta, \qquad r_\nu = \log_{10} \nu \tag{6.4}$$

before affine scaling. Equal increments in r_η then represent equal multiplicative changes in η. The logarithm is defined only for positive values; an ideal-limit case with $\eta = 0$ requires a separate representation or a declared floor and cannot be silently passed through $\log_{10}$.

The transformation must be chosen from the actual parameter design. A narrow linear range does not automatically benefit from a logarithm.

6.4 Output centering and scale balance

The magnitudes and variances of ψ and U may differ considerably. An unscaled sum of squared errors can then be dominated by the larger-amplitude field. The recommended primary baseline standardizes the two outputs separately using training-set statistics:

$$\widetilde{\psi} = \frac{\psi - m_\psi}{s_\psi}, \qquad \widetilde{U} = \frac{U - m_U}{s_U}. \tag{6.5}$$

The network predicts $(\widehat{\widetilde{\psi}}, \widehat{\widetilde{U}})$, and physical dimensionless predictions are reconstructed as

$$\widehat{\psi} = m_\psi + s_\psi \widehat{\widetilde{\psi}}, \qquad \widehat{U} = m_U + s_U \widehat{\widetilde{U}}. \tag{6.6}$$

Global training statistics can underrepresent small-amplitude early dynamics or localized current sheets. This is a limitation to evaluate, not a reason to compute statistics from the test set. Alternative scale definitions may be compared on validation cases and then frozen.

6.5 Leakage-free preprocessing

Let $\mathcal{D}_{\mathrm{tr}}$ denote only the training observations. Every learned statistic must have the form

$$(m_a, s_a) = \mathrm{fitScale}(\mathcal{D}_{\mathrm{tr}}). \tag{6.7}$$

The same stored values are then applied to validation and test inputs. Recomputing a mean or range separately on each held-out case gives the model information about that case and can make deployment requirements unrealistic.

6.6 Gauge and mean conventions

The project fixes the spatial mean of U to zero. A constant shift in U changes neither velocity nor vorticity, but it does change a direct U regression loss. Every reference case must therefore use the same gauge. Likewise, the mean of ψ must be recorded and kept consistent with the dataset convention. If cases use arbitrary additive offsets, the network wastes capacity learning nonphysical bookkeeping differences.

7 Representing the periodic spatial domain

7.1 Why raw coordinates create an artificial seam

The domain is periodic:

$$(x, y) \in [0, L_x) \times [0, L_y). \tag{7.1}$$

Physically, $x = 0$ and $x = L_x$ describe the same location. Raw scalar coordinates represent them as far apart. A generic multilayer perceptron has no inherent knowledge that the endpoints must join.

7.2 Sine-cosine periodic encoding

Define phase angles

$$\phi_x = \frac{2\pi x}{L_x}, \qquad \phi_y = \frac{2\pi y}{L_y}. \tag{7.2}$$

The basic periodic feature map is

$$\boldsymbol{e}_{\mathrm{per}}(x, y) = \begin{bmatrix} \sin\phi_x & \cos\phi_x & \sin\phi_y & \cos\phi_y \end{bmatrix}^{\mathsf{T}}. \tag{7.3}$$

Because sine and cosine repeat every 2π,

$$\boldsymbol{e}_{\mathrm{per}}(0, y) = \boldsymbol{e}_{\mathrm{per}}(L_x, y) \tag{7.4}$$

and similarly in y. Any deterministic network of these features has equal endpoint values. This is a *hard representation property*, not a penalty added to the loss.

The recommended processed input is therefore

$$\boldsymbol{z} = \begin{bmatrix} \sin\phi_x, \cos\phi_x, \sin\phi_y, \cos\phi_y, \widetilde{t}, \widetilde{r}_\eta, \widetilde{r}_\nu, \widetilde{A}_0 \end{bmatrix}^{\mathsf{T}}, \tag{7.5}$$

with raw or logarithmic coefficient transforms selected and frozen as described in Section 6. Equation (7.5) contains eight numerical components even though the physical input has six variables.

7.3 Value periodicity versus derivative periodicity

Smooth functions of the sine-cosine encoding are periodic together with their derivatives. However, numerical floating-point evaluation, non-smooth activations, or later implementation mistakes can still produce derivative artifacts. Periodic values and required derivatives should be verified explicitly on evaluation grids.

7.4 Higher harmonics and Fourier features

Thin current sheets and nonlinear structures contain higher spatial frequencies than a single sine-cosine pair. One deterministic extension uses harmonics

$$\sin(n\phi_x),\ \cos(n\phi_x),\ \sin(n\phi_y),\ \cos(n\phi_y), \qquad n = 1, \dots, N_h. \tag{7.6}$$

Random or learned Fourier-feature maps are another option and can reduce the tendency of coordinate multilayer perceptrons to learn low spatial frequencies first [6]. They also introduce a frequency-scale hyperparameter and can create overly oscillatory fits. The primary baseline should begin with the basic periodic encoding and treat richer features as a validation-selected ablation.

Recommended practice

Periodic encoding is part of a competent baseline, not a physics-loss advantage reserved for the PINN. Module 4 and Module 5 should use the same coordinate representation unless the purpose of an explicit architecture ablation is stated.

8 The multilayer perceptron

8.1 Neuron, layer, weight, and bias

A *multilayer perceptron* (MLP) is a feedforward neural network made from repeated affine transformations and nonlinear activation functions. *Feedforward* means information moves from input to output without a recurrent state loop.

Let $\boldsymbol{h}^{(0)} = \boldsymbol{z}$ be the processed input. For hidden layer $\ell = 1, \dots, L$, define

$$\boldsymbol{a}^{(\ell)} = \boldsymbol{W}^{(\ell)}\boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}, \tag{8.1}$$

$$\boldsymbol{h}^{(\ell)} = \sigma\left(\boldsymbol{a}^{(\ell)}\right). \tag{8.2}$$

Here

- $\boldsymbol{W}^{(\ell)}$ is a matrix of trainable *weights*;
- $\boldsymbol{b}^{(\ell)}$ is a vector of trainable *biases*;
- $\boldsymbol{a}^{(\ell)}$ is the pre-activation vector;
- $\boldsymbol{h}^{(\ell)}$ is the hidden representation; and
- σ is applied component by component.

The output layer is usually linear for regression:

$$\widehat{\widetilde{\boldsymbol{q}}} = \boldsymbol{W}^{(L+1)}\boldsymbol{h}^{(L)} + \boldsymbol{b}^{(L+1)}, \qquad \widehat{\widetilde{\boldsymbol{q}}} = \begin{bmatrix} \widehat{\widetilde{\psi}} & \widehat{\widetilde{U}} \end{bmatrix}^{\mathsf{T}}. \tag{8.3}$$

8.2 Why nonlinear activations are essential

If $\sigma(a) = a$ in every layer, composing layers produces another affine map. Depth would add no nonlinear expressive power. A nonlinear activation lets the network represent curved and interacting dependence on coordinates, time, and parameters.

Common activations include

$$\tanh(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}, \tag{8.4}$$

$$\mathrm{ReLU}(a) = \max(0, a), \tag{8.5}$$

$$\mathrm{SiLU}(a) = \frac{a}{1 + e^{-a}}, \tag{8.6}$$

$$\sin(a) = \text{sine activation used in some implicit representations.} \tag{8.7}$$

The hyperbolic tangent is smooth to all derivative orders and is therefore a natural default for the later PINN, which needs high-order derivatives. ReLU is continuous but its classical second derivative is zero away from its kink and undefined at the kink, making it unsuitable for a strong-form residual requiring Laplacians. To keep the architecture comparable across Modules 4 and 5, the primary baseline uses tanh.

8.3 Depth, width, and capacity

The *depth* is the number of hidden layers L. The *width* n_ℓ is the number of units in layer ℓ. *Capacity* informally describes how rich a function class the architecture can express. Universal approximation results show that certain neural networks can approximate broad classes of continuous functions on compact domains, but these existence theorems do not guarantee efficient training, low data requirement, reliable derivatives, or extrapolation [3].

If the processed input has dimension n_0, hidden widths are $n_1, \ldots, n_L$, and output dimension is $n_{L+1} = 2$, the number of trainable scalar parameters is

$$N_\theta = \sum_{\ell=1}^{L+1} \left(n_\ell n_{\ell-1} + n_\ell\right). \tag{8.8}$$

The first term counts weights and the second counts biases.

For an eight-component processed input, six hidden layers of width 128, and two outputs,

$$N_\theta = (128 \cdot 8 + 128) + 5(128 \cdot 128 + 128) + (2 \cdot 128 + 2) \tag{8.9}$$

$$= 84{,}738. \tag{8.10}$$

The exact count should be logged automatically because architectural details can otherwise be described inconsistently.

8.4 Shared trunk and output heads

The simplest network uses a shared hidden representation and one two-component linear output. An alternative uses a shared *trunk* followed by separate small *heads* for ψ and U. A shared trunk can learn common spatiotemporal features, while separate heads reduce direct competition between outputs. The head design is a hyperparameter; Module 4 and Module 5 should share it in the primary comparison.

8.5 Residual connections and normalization layers

A residual connection maps

$$\boldsymbol{h}^{(\ell+1)} = \boldsymbol{h}^{(\ell)} + F_\ell(\boldsymbol{h}^{(\ell)}), \tag{8.11}$$

which can improve gradient flow in deeper networks. It is unrelated to a PDE residual despite the shared word "residual."

Batch normalization makes predictions depend on batch statistics and can complicate deterministic coordinate derivatives. Dropout randomly masks activations during training and changes the effective function. Neither is necessary for the primary smooth coordinate-network baseline. Their use would require careful train/evaluation handling and a fair counterpart in Module 5.

8.6 Weight initialization

Training begins from random weights. If their variance is too large, activations and gradients can explode or saturate; if too small, signals can vanish. Xavier or Glorot initialization scales weights using input and output widths and is a standard choice for tanh networks [4]. Random seeds must be recorded, but one seed is not evidence of robustness.

9 Forward prediction as an implicit field representation

9.1 One point

Given $(x, y, t, \eta, \nu, A_0)$:

1. validate that the values lie in the declared deployment domain;

2. construct periodic spatial features;
3. apply stored input transforms;
4. evaluate the network equations layer by layer;
5. invert output standardization; and
6. return $(\widehat{\psi}, \widehat{U})$ with metadata identifying the model and convention.

9.2 A full field snapshot

To reconstruct a snapshot at fixed $(t, \boldsymbol{\mu})$, evaluate the network at every desired (x_i, y_j) and reshape the predictions into $N_x \times N_y$ arrays. This operation is parallel across points. A full trajectory repeats it for multiple times.

The coordinate network does not store a mesh internally. Nevertheless, the cost of producing a full field scales with the number of queried coordinates. Reporting only the cost of one point prediction is therefore misleading when the reference task requires a complete field.

9.3 Parameter conditioning

Because η, ν, and A_0 enter every point from a case, the network must learn how the spatial-temporal field changes with these case-level inputs. If there are few distinct cases, the network may memorize their parameter labels rather than learn smooth dependence. Dense point sampling cannot compensate completely for sparse coverage of parameter space.

9.4 Spectral bias

Coordinate MLPs trained by gradient descent often fit low-frequency components before high-frequency components. This tendency is called *spectral bias* or *frequency bias*. In island coalescence, broad flux structures may therefore be learned before a thin current sheet. Fourier features, adaptive sampling, and architecture changes may help, but each changes the experimental design. Derived-current error is an especially sensitive indicator.

10 The strict data-only objective

10.1 Pointwise error

For supervised example n, define standardized errors

$$e_{\psi,n} = \widehat{\widetilde{\psi}}_n - \widetilde{\psi}_n, \qquad e_{U,n} = \widehat{\widetilde{U}}_n - \widetilde{U}_n. \tag{10.1}$$

The mean squared error over a set $\mathcal{B}$ of $N_{\mathcal{B}}$ examples is

$$\mathrm{MSE}_a(\mathcal{B}) = \frac{1}{N_{\mathcal{B}}} \sum_{n \in \mathcal{B}} e_{a,n}^2, \qquad a \in \{\psi, U\}. \tag{10.2}$$

10.2 Channel-balanced data loss

The primary value-only objective is

$$\boxed{\mathcal{L}_{\mathrm{data}} = \lambda_\psi \, \mathrm{MSE}_\psi + \lambda_U \, \mathrm{MSE}_U,} \tag{10.3}$$

with $\lambda_\psi = \lambda_U = 1$ after separate output standardization. Equal weights then give the two standardized fields equal explicit importance. If different weights are used, their selection and physical consequence must be reported.

The strict Module 4 training objective is

$$\boxed{\mathcal{L}_{\mathrm{train}} = \mathcal{L}_{\mathrm{data}}.} \tag{10.4}$$

There is no $\mathcal{L}_{\mathrm{PDE}}$, $\mathcal{L}_{\mathrm{IC}}$, $\mathcal{L}_{\mathrm{BC}}$, or $\mathcal{L}_{\mathrm{global}}$ term.

10.3 Case-balanced empirical risk

If case c contributes N_c points, a simple global point average weights that case in proportion to N_c. This is undesirable when different cases have different saved resolutions or sampling densities. A case-balanced loss first averages within each selected case and then across cases:

$$\mathcal{L}_{\mathrm{data}} = \frac{1}{|\mathcal{C}_{\mathcal{B}}|} \sum_{c \in \mathcal{C}_{\mathcal{B}}} \left[\lambda_\psi \frac{1}{N_c} \sum_{n \in c} e_{\psi,n}^2 + \lambda_U \frac{1}{N_c} \sum_{n \in c} e_{U,n}^2 \right], \tag{10.5}$$

where $\mathcal{C}_{\mathcal{B}}$ is the set of cases represented in the batch. A hierarchical batch sampler can select cases uniformly and then select points uniformly within each chosen case.

10.4 Why squared error is reasonable and limited

Mean squared error penalizes large errors strongly, is differentiable, and corresponds to maximum likelihood under an independent Gaussian error model with constant variance. Solver samples are not truly independent Gaussian measurements, so this probabilistic interpretation is only an approximation. MSE can also underweight localized structures occupying a small fraction of the domain.

Mean absolute error,

$$\mathrm{MAE}_a = \frac{1}{N}\sum_{n=1}^{N} |e_{a,n}|, \tag{10.6}$$

is less sensitive to large outliers but has a non-differentiable point at zero. A Huber loss transitions between squared and absolute behavior. These are validation-selected alternatives, not reasons to change the primary baseline after observing test results.

10.5 Are gradient or spectral losses still data-only?

A loss comparing predicted gradients to gradients computed from reference data uses no PDE and is therefore data-supervised in a broad sense. A spectral loss comparing Fourier coefficients is also data-supervised. However, either provides more information per observation than the strict point-value loss and changes the comparison with Module 5. They should be labeled as separate enhanced-data baselines, with the primary baseline retained.

Frozen project convention

The primary Module 4 loss uses only observed values of ψ and U. No target J, target ω, spatial-gradient target, Fourier-coefficient target, PDE residual, initial-condition residual, boundary residual, or energy residual updates the network.

10.6 Regularization is not physical information

Weight decay adds

$$\mathcal{L}_{\mathrm{wd}} = \alpha \sum_k \theta_k^2 \tag{10.7}$$

or applies an equivalent decoupled parameter shrinkage. This is a numerical or statistical regularizer, not an MHD constraint. If used, α must be chosen on validation cases and matched or explicitly accounted for in Module 5.

11 Backpropagation and optimization

11.1 What training changes

The loss is a scalar function of all trainable parameters:

$$\mathcal{L}_{\mathrm{train}} = \mathcal{L}_{\mathrm{train}}(\boldsymbol{\theta}). \tag{11.1}$$

The gradient

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathrm{train}} = \begin{bmatrix} \partial \mathcal{L}_{\mathrm{train}}/\partial\theta_1 & \cdots & \partial \mathcal{L}_{\mathrm{train}}/\partial\theta_{N_\theta} \end{bmatrix}^{\mathsf{T}} \tag{11.2}$$

points in the direction of greatest local increase. Gradient descent takes a step in the opposite direction:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha_k \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\text{train}}(\boldsymbol{\theta}_k), \tag{11.3}$$

where $\alpha_k > 0$ is the learning rate at update k.

11.2 Backpropagation

Backpropagation is an efficient application of the chain rule through the network's computational graph. It propagates the sensitivity of the loss from the outputs backward through each layer to all weights and biases. Backpropagation is not itself an optimizer; it computes derivatives used by an optimizer.

For a scalar chain $z \mapsto a \mapsto h \mapsto \mathcal{L}$,

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial h} \frac{\partial h}{\partial a} \frac{\partial a}{\partial z}. \tag{11.4}$$

The matrix-valued network uses the same principle with Jacobians and vector-Jacobian products. Modern frameworks perform this calculation by automatic differentiation.

11.3 Full-batch, stochastic, and mini-batch gradients

A full-batch update uses every training point, which may be prohibitively expensive. Stochastic gradient descent uses one randomly selected example. Mini-batch training uses a subset and estimates the full gradient:

$$\boldsymbol{g}_{\mathcal{B}} = \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{B}}. \tag{11.5}$$

The estimate is noisy, but many inexpensive updates can be effective. For correlated field data, hierarchical selection of cases, times, and points provides better control than treating all flattened rows as exchangeable.

11.4 Batch, update, and epoch

- A **batch** is the set of examples used in one gradient evaluation.
- An **update** changes the parameters once.
- An **epoch** conventionally means enough batches to process approximately one pass through a fixed training set.

If points are sampled continuously or resampled from large arrays, "epoch" may be ambiguous. The project should record the exact number of optimizer updates and examples processed in addition to any epoch count.

11.5 The Adam optimizer

Adam maintains exponentially weighted estimates of the first and second moments of the gradient [5]. In simplified componentwise form,

$$\boldsymbol{m}_k = \beta_1 \boldsymbol{m}_{k-1} + (1-\beta_1)\boldsymbol{g}_k, \tag{11.6}$$

$$\boldsymbol{v}_k = \beta_2 \boldsymbol{v}_{k-1} + (1-\beta_2)\boldsymbol{g}_k \odot \boldsymbol{g}_k, \tag{11.7}$$

where $\odot$ denotes elementwise multiplication. Bias-corrected estimates $\widehat{\boldsymbol{m}}_k$ and $\widehat{\boldsymbol{v}}_k$ are used in

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha \frac{\widehat{\boldsymbol{m}}_k}{\sqrt{\widehat{\boldsymbol{v}}_k} + \epsilon}, \tag{11.8}$$

with division, square root, and addition applied componentwise. Adam is a practical default, not a guarantee of reaching the best minimum.

11.6 Learning-rate schedules

If the learning rate is too large, loss may oscillate or diverge. If too small, training can stagnate. A schedule may reduce α after a fixed number of updates, after validation stagnation, or according to a smooth cosine curve. The schedule and trigger must be logged. Changing it based on the test set is leakage.

11.7 Early stopping and checkpoints

Training loss often continues decreasing after validation error begins increasing. This is *overfitting*. Early stopping saves the parameter state with the best declared validation metric and stops after a patience interval without improvement. The final test must use that frozen checkpoint, not the last update or the test-optimal update.

11.8 Randomness and repeated training

Random initialization, batch order, and possibly accelerator operations cause run-to-run variation. At least several independent seeds should be trained for the final comparison. Report mean, spread, and individual failures rather than only the best seed. The same seed values should be paired across Module 4 and Module 5 where technically meaningful.

12 Sampling observations from the training cases

12.1 Why not train on every stored point at every update?

A 128×128 grid with 101 stored times contains about 1.65 million points per case. A parameter sweep can therefore contain tens or hundreds of millions of point records. Mini-batch sampling reduces memory use and enables controlled data-budget experiments.

12.2 Hierarchical sampling

A defensible batch can be constructed by:

1. selecting B_c training cases uniformly;
2. selecting B_t stored times within each case;
3. selecting B_s spatial coordinates within each selected snapshot; and
4. returning $B_c B_t B_s$ point examples.

This makes the unit of physical diversity explicit. Sampling all points from one case for many updates can produce highly correlated gradients and temporarily neglect other parameter combinations.

12.3 Uniform and structure-aware sampling

Uniform spatial sampling estimates domain-average point error. It can miss a thin current sheet because the sheet occupies little area. A mixture distribution may combine uniform points with points drawn from regions of high reference $|J|$, large field gradient, or reconnection activity.

Structure-aware sampling uses reference-derived information and therefore changes the information supplied to the learner. It remains data-only if no PDE residual is used, but it must be applied identically to Module 4 and the data term of Module 5 for a fair comparison. Evaluation must remain on an unbiased full grid, and importance weights are needed if the training loss is intended to estimate a uniform-domain risk.

12.4 Time sampling

Uniform sampling over stored times may overrepresent slowly varying phases if the output cadence is uniform but the event of interest is brief. Alternatives include stratification into early, reconnection, and late-time windows. Any event-aware definition must be computed from training cases and frozen before test evaluation.

12.5 Data budget has several meanings

"Ten thousand training points" is incomplete. Data budget can refer to:

- number of independently simulated parameter cases;
- number of observed snapshots per case;
- number of observed spatial points per snapshot;
- number of output channels observed; and
- total reference-solver cost required to produce those observations.

The project should vary at least the number of training cases and the number of point observations per case. Physics information may help in one scarcity regime but not another.

13 Hyperparameters and model selection

13.1 Trainable parameters versus hyperparameters

Weights and biases are trainable parameters updated by backpropagation. Width, depth, activation, batch size, learning rate, sampling mixture, output scaling, and early-stopping patience are hyperparameters selected outside those updates.

13.2 A staged tuning strategy

An efficient, scientifically controlled sequence is:

1. verify data loading and inverse transforms on a tiny subset;
2. overfit one snapshot to confirm network and loss correctness;
3. overfit one complete case to confirm time and coordinate handling;
4. train a small multi-case model and inspect validation behavior;
5. tune a limited predeclared set of architecture and optimization choices;
6. freeze the configuration; and
7. train repeated seeds and evaluate the untouched test cases.

The ability to overfit a tiny dataset is a debugging test, not evidence of generalization.

13.3 Validation metric selection

Selecting checkpoints solely by standardized ψ/U MSE may favor broad smooth structures and neglect current sheets. A predeclared validation score can combine primary-field relative errors and derived-field diagnostics. However, if a derived-current score determines model selection, it has influenced training decisions even if it is absent from gradient updates. That is acceptable for the baseline, but it must be disclosed and applied consistently to Module 5.

13.4 Search budget fairness

One method should not receive vastly more manual tuning than the other. Report the hyperparameter search space, number of trials, selection rule, and total compute. Fairness does not require identical optimal hyperparameters, but it does require comparable opportunity and transparent accounting.

14 Evaluation on complete unseen fields

14.1 Why training loss is not the final metric

Training loss measures fit to observed training points in transformed units. Scientific performance concerns physical dimensionless fields on complete held-out cases. Predictions must be inverse-transformed and evaluated on full grids and times that include points not used as observations.

14.2 Absolute error fields

For field $a \in \{\psi, U, J, \omega\}$,

$$e_a(x, y, t; \boldsymbol{\mu}) = \widehat{a} - a. \tag{14.1}$$

Plotting e_a with a symmetric color scale reveals where the surrogate differs. Error plots must accompany predicted contours because visually similar fields can have structured phase or amplitude errors.

14.3 Discrete norms

On an evaluation grid with N points, define

$$\|e_a\|_2 = \left(\sum_{n=1}^{N} e_{a,n}^2 \right)^{1/2}, \tag{14.2}$$

$$\|e_a\|_\infty = \max_{1 \le n \le N} |e_{a,n}|. \tag{14.3}$$

The relative L_2 error is

$$\epsilon_{2,a} = \frac{\|\widehat{\boldsymbol{a}} - \boldsymbol{a}\|_2}{\|\boldsymbol{a}\|_2 + \epsilon_{\text{floor}}}, \tag{14.4}$$

where ϵ_{floor} prevents division by zero. The floor must be declared; otherwise relative error becomes arbitrarily large when the true field norm is near zero.

The root mean squared error is

$$\text{RMSE}_a = \sqrt{\frac{1}{N}\sum_{n=1}^{N} e_{a,n}^2}. \tag{14.5}$$

Report both dimensional-status-aware absolute metrics and relative metrics. A small relative error in a dominant background can coexist with a large error in a physically important perturbation.

14.4 Per-case and aggregate reporting

Compute metrics separately for every held-out case and time. Then report median, mean, quantiles, and worst case across cases. Pooling all points before computing one metric weights larger cases more heavily and can hide complete failure on one parameter combination.

14.5 Time-resolved error

For case c, define

$$\epsilon_{2,a}^{(c)}(t_k) = \frac{\|\widehat{a}^{(c)}(\cdot,\cdot,t_k) - a^{(c)}(\cdot,\cdot,t_k)\|_2}{\|a^{(c)}(\cdot,\cdot,t_k)\|_2 + \epsilon_{\text{floor}}}. \tag{14.6}$$

This curve distinguishes an early-time fit from failure near current-sheet formation or late-time decay. A single time-averaged number cannot show when the model loses fidelity.

14.6 Phase error versus amplitude error

A slight spatial displacement of a narrow structure can produce large pointwise error even when its amplitude and shape are reasonable. Conversely, a blurred structure can have moderate MSE while missing the correct peak. Report peak location, peak magnitude, and physically defined reconnection diagnostics in addition to field norms.

15 Physical diagnostics that do not train Module 4

15.1 The distinction between loss and diagnostic

A *loss term* contributes a gradient that changes $\boldsymbol{\theta}$. A *diagnostic* is computed for analysis after or between training updates but does not contribute to those updates. The same mathematical quantity can serve either role depending on how it is used.

This distinction allows Module 4 to be evaluated physically without becoming physics-informed.

15.2 Derived current and vorticity

For a smooth coordinate network, automatic differentiation can compute

$$\widehat{J} = -\left(\frac{\partial^2 \widehat{\psi}}{\partial x^2} + \frac{\partial^2 \widehat{\psi}}{\partial y^2}\right), \qquad \widehat{\omega} = \frac{\partial^2 \widehat{U}}{\partial x^2} + \frac{\partial^2 \widehat{U}}{\partial y^2}. \tag{15.1}$$

The derivatives must be with respect to physical dimensionless x and y, including all periodic-encoding and output-scaling chain rules. A useful cross-check computes derivatives both by automatic differentiation and by the same spectral differentiation applied to network predictions on a uniform grid.

15.3 Magnetic and kinetic energies

With the Module 1 convention,

$$E_B(t) = \frac{1}{2}\int_\Omega |\nabla \psi|^2 \, \mathrm{d}A, \tag{15.2}$$

$$E_K(t) = \frac{1}{2}\int_\Omega |\nabla U|^2 \, \mathrm{d}A. \tag{15.3}$$

Compute predicted histories $\widehat{E}_B$ and $\widehat{E}_K$ from predicted fields and compare them with reference histories. Energy agreement tests global field gradients and is not guaranteed by pointwise value MSE.

15.4 Peak current and reconnection quantities

Current-sheet fidelity can be measured by

$$J_{\max}(t) = \max_{(x,y)\in\Omega} |J(x,y,t)|. \tag{15.4}$$

Report relative peak error and location error. Reconnected flux and reconnection rate must use the versioned O-point/X-point definition established by Modules 2 and 6. A surrogate should not be credited for visual island formation if these quantitative histories are wrong.

15.5 Periodicity defects

For a field a, value mismatch across the x boundary may be summarized as

$$\epsilon^{(a)}_{\mathrm{per,x}}(t) = \frac{\|a(0,\cdot,t) - a(L_x,\cdot,t)\|_2}{\|a(0,\cdot,t)\|_2 + \epsilon_{\mathrm{floor}}}, \tag{15.5}$$

with analogous y and derivative defects. The sine-cosine representation should make value mismatch very small by construction, so a large defect indicates an implementation error.

15.6 PDE residuals as evaluation-only quantities

The frozen reduced-MHD residuals are

$$\mathcal{R}_\psi = \partial_t \widehat{\psi} + \left[\widehat{U}, \widehat{\psi}\right] - \eta \nabla^2 \widehat{\psi}, \tag{15.6}$$

$$\mathcal{R}_\omega = \partial_t \widehat{\omega} + \left[\widehat{U}, \widehat{\omega}\right] - \left[\widehat{J}, \widehat{\psi}\right] - \nu \nabla^2 \widehat{\omega}. \tag{15.7}$$

The Poisson bracket is

$$[f, g] = f_x g_y - f_y g_x. \tag{15.8}$$

Computing these residuals after training measures whether data fit happens to yield a PDE-consistent continuous field. They must not be backpropagated into the Module 4 model.

Residual magnitude depends on normalization, derivative order, and evaluation distribution. Report a dimensionless or term-normalized residual together with the raw residual and the magnitudes of the individual PDE terms. A small cancellation-based residual can otherwise be misread.

15.7 Energy-balance defect

For the unforced periodic system, Module 1 gives

$$\frac{\mathrm{d}}{\mathrm{d}t}(E_B + E_K) = -\eta \int_\Omega J^2 \,\mathrm{d}A - \nu \int_\Omega \omega^2 \,\mathrm{d}A. \tag{15.9}$$

Define the predicted balance defect

$$\mathcal{D}_E(t) = \frac{\mathrm{d}}{\mathrm{d}t}(\widehat{E}_B + \widehat{E}_K) + \eta \int_\Omega \widehat{J}^2 \,\mathrm{d}A + \nu \int_\Omega \widehat{\omega}^2 \,\mathrm{d}A. \tag{15.10}$$

Again, this is a diagnostic in Module 4 and an optional training constraint only in Module 5.

Learning checkpoint

A data-only network can have low field MSE and large PDE or energy-balance defects. That observation is not an inconsistency in the experiment; it is precisely one of the phenomena that the Module 4 versus Module 5 comparison is designed to measure.

16 Data efficiency and learning curves

16.1 What data efficiency means

A model is more data-efficient if it reaches a specified generalization performance using fewer observed data, holding relevant conditions fixed. Because data have hierarchical structure, efficiency must be plotted against more than one budget axis.

Recommended experiments include:

1. fix points per case and vary the number of training cases;
2. fix the case set and vary observed snapshots per case;
3. fix cases and snapshots and vary spatial points per snapshot; and
4. compare uniform sampling with a declared structure-aware mixture.

For budget B, train repeated random seeds and evaluate the same fixed validation or test cases. A learning curve plots a metric such as median held-out relative L_2 error against B, preferably on logarithmic axes when budgets span orders of magnitude.

16.2 Reference-generation cost matters

If one complete numerical trajectory must be run before any points from that case are available, then reducing point observations may not reduce solver cost. Distinguish stored-data volume from expensive simulation-case count. The project should report both.

16.3 No single accuracy threshold is universal

The significance of a field error depends on downstream use. A model used for qualitative visualization can tolerate errors that would be unacceptable for peak-current prediction or stability thresholds. Data-efficiency claims must name the metric and target threshold.

17 Overfitting, underfitting, and generalization failure

17.1 Underfitting

Underfitting occurs when the model cannot fit even the training data adequately. Possible causes include insufficient capacity, poor scaling, an overly strong regularizer, inadequate optimization, or input features incapable of representing periodic or high-frequency structure.

Typical evidence is high training and validation error that plateau together. Increasing width or training duration may help, but only after verifying the pipeline.

17.2 Overfitting

Overfitting occurs when training error is small but held-out error remains large or worsens. The model has adapted to peculiarities of the observed examples that do not transfer. More independent cases, regularization, early stopping, or a more appropriate inductive bias may help.

17.3 Parameter memorization

When only a few discrete parameter combinations are present, a high-capacity network can behave like a collection of case-specific fits keyed by (η, ν, A_0). Good reconstruction on random held-out points from those same cases does not reveal this. Whole-case tests do.

17.4 Loss domination by easy regions

Most domain points may lie far from a thin current sheet. A low uniform MSE can therefore coexist with poor reconnection dynamics. Derived-field metrics and stratified error reports expose this. Changing sampling can help, but must remain controlled across models.

17.5 Gauge contamination

If U has arbitrary case-dependent constant offsets, a direct regression loss treats those offsets as real signal. The model can have high U error while predicting velocity accurately, or low U error by learning irrelevant offsets. Freeze $\langle U \rangle = 0$ before training.

17.6 Boundary seam artifacts

Raw coordinate inputs may fit both sides of a periodic boundary separately and create a seam. Periodic encoding prevents the most direct value mismatch and should be verified with derivative checks.

17.7 Derivative amplification

Suppose the prediction error contains a Fourier component

$$e_\psi = \epsilon \cos(k_x x). \tag{17.1}$$

Its amplitude is ϵ, but its contribution to current error is

$$e_J = -\nabla^2 e_\psi = \epsilon k_x^2 \cos(k_x x). \tag{17.2}$$

The curvature error grows as k_x^2. This explains how a small high-frequency flux error can dominate current error.

17.8 Distribution shift

A test case may differ from training not only in parameter values but also in grid, output cadence, numerical solver version, normalization, initial-condition formula, or physical regime. Some shifts are desired tests; others are schema incompatibilities. The manifest must distinguish them.

17.9 Optimization failure masquerading as model failure

A poor seed, unstable learning rate, or saturated activation can make one run fail. Repeated seeds and training-curve inspection separate optimization reliability from representational limitation.

17.10 Visual plausibility

Contour plots are necessary but not sufficient. Color normalization can hide amplitude errors; broad structures can conceal high-frequency noise; and a misplaced current sheet can still look qualitatively reasonable. Every visual comparison must be paired with quantitative metrics and identical color ranges.

18 Fair comparison with the Module 5 PINN

18.1 What should be held fixed

The primary comparison should hold fixed:

- training, validation, and test case manifests;
- observed ψ/U points and data budgets;
- input and output transformations;
- periodic coordinate encoding;
- architecture family and approximately the same parameter count;
- initialization scheme and repeated seed set;
- data-loss definition and data-batch sampler;
- validation metrics and final evaluation grids; and

- reporting format.

Module 5 then adds collocation points and physics losses. It may require different optimizer settings because the objective is harder, but those differences and their tuning budgets must be reported.

18.2 Three distinct notions of fairness

No single control makes every comparison fair.

Fairness axis	Held equal	Scientific question
Observed-data fairness	Number and identity of labeled field observations	Does physics reduce dependence on labeled simulation values?
Architecture fairness	Network family and parameter count	Does the loss information help without extra representational capacity?
Compute fairness	Wall time, updates, or accelerator budget	Which method performs better for a fixed computational investment?

The primary scientific comparison should prioritize equal labeled data and comparable architecture. Compute must still be reported because automatic differentiation of high-order PDE residuals can be expensive. A secondary compute-matched comparison is valuable.

18.3 Matched observation budgets

For every data budget B, construct one immutable observation index list. Both models receive exactly those target values. The PINN may additionally receive unlabeled collocation coordinates, but the number and sampling of those coordinates must be logged. Otherwise, it is unclear whether improvement arose from physics or simply from evaluating many more locations.

18.4 Ablation ladder

A clean experiment grows the Module 4 objective in stages:

$$\text{A: } \mathcal{L}_{\text{data}}, \tag{18.1}$$

$$\text{B: } \mathcal{L}_{\text{data}} + w_{\text{PDE}}\mathcal{L}_{\text{PDE}}, \tag{18.2}$$

$$\text{C: } \mathcal{L}_{\text{data}} + w_{\text{PDE}}\mathcal{L}_{\text{PDE}} + w_{\text{IC}}\mathcal{L}_{\text{IC}}, \tag{18.3}$$

$$\text{D: } \mathcal{L}_{\text{data}} + w_{\text{PDE}}\mathcal{L}_{\text{PDE}} + w_{\text{IC}}\mathcal{L}_{\text{IC}} + w_{\text{BC}}\mathcal{L}_{\text{BC}}, \tag{18.4}$$

$$\text{E: D} + w_{\text{global}}\mathcal{L}_{\text{global}}. \tag{18.5}$$

This ladder identifies which information source changes performance. Module 4 is model A; models B–E belong to Module 5.

18.5 What would count as evidence that physics helped?

Evidence includes statistically consistent improvement over repeated seeds on predeclared held-out cases, especially at reduced labeled-data budgets, together with improved physical diagnostics and transparent compute cost. One favorable contour or one best seed is insufficient.

19 Recommended primary baseline specification

The following is a defensible starting configuration, not a claim that it is universally optimal.

Frozen project convention : version-1 Module 4 baseline

- **Physical map:** $(x, y, t, \eta, \nu, A_0) \mapsto (\widehat{\psi}, \widehat{U})$.
- **Spatial representation:** sine-cosine periodic encoding in x and y.
- **Other inputs:** affine scaling of t and A_0; use $\log_{10}\eta$ and $\log_{10}\nu$ only if the parameter design spans positive orders of magnitude, otherwise scale raw coefficients.
- **Outputs:** separate training-set standardization of ψ and U; inverse-transform before scientific evaluation.
- **Architecture:** fully connected MLP with six hidden layers, width 128, tanh activation, and a linear two-output head.
- **Initialization:** Xavier/Glorot initialization appropriate to tanh.
- **Training loss:** case-balanced sum of standardized ψ and U MSE with equal channel weights.
- **Forbidden training terms:** PDE, IC, BC, global balance, J, ω, derivative, and spectral target losses.
- **Optimizer:** Adam with a validation-selected learning-rate schedule and optional validation-selected weight decay.
- **Batches:** hierarchical case-time-space sampling.
- **Splits:** immutable whole-case train/validation/test manifests, with interpolation and extrapolation test groups separated.
- **Selection:** best checkpoint by a predeclared validation score; test set untouched until configuration freeze.
- **Replicates:** multiple documented random seeds.
- **Evaluation:** complete-grid fields, derived J/ω, energies, peak current, reconnection diagnostics, periodicity, and evaluation-only PDE residuals.

19.1 Why this is only a starting point

The correct width and depth depend on the verified solution complexity and parameter sweep. The configuration must first pass small-data debugging, then be tuned on validation cases. The frozen comparison configuration should be written to a version-controlled file rather than existing only in a notebook or command history.

19.2 Suggested controlled ablations

After the primary baseline is frozen, useful one-change-at-a-time studies include:

- raw coordinates versus periodic encoding;
- basic periodic encoding versus higher Fourier harmonics;
- shared output layer versus separate output heads;
- tanh versus another smooth activation;
- uniform versus structure-aware data sampling;
- unscaled versus separately standardized outputs; and
- strict value-only loss versus an explicitly labeled derivative-data baseline.

These studies should not be mixed indiscriminately into the PINN comparison.

20 Worked miniature example

20.1 A simple decaying magnetic mode

Module 1 introduced the exact solution

$$\psi(x, y, t; \eta, A) = Ae^{-\eta(k_x^2+k_y^2)t} \cos(k_x x) \cos(k_y y), \qquad U = 0. \tag{20.1}$$

Although this is much simpler than island coalescence, it illustrates the complete supervised map. Take ν as an inactive input and $A_0 = A$. Each example is

$$\boldsymbol{s}_n = (x_n, y_n, t_n, \eta_n, \nu_n, A_n), \tag{20.2}$$

$$\boldsymbol{q}_n = (\psi_n, 0). \tag{20.3}$$

The network is not given the exponential or cosine formula. It observes values at selected combinations and minimizes Eq. (10.3).

20.2 What different tests mean

If the network is trained at $\eta \in \{10^{-3}, 3 \times 10^{-3}\}$ and tested at 2×10^{-3}, it tests interpolation in η. Testing at 10^{-2} is extrapolation. If training includes $0 \le t \le 5$ and testing uses unused points within that interval, it tests time interpolation. Testing at $t = 8$ tests temporal extrapolation.

20.3 Why the PDE diagnostic can fail despite a good fit

The exact solution satisfies

$$\partial_t \psi - \eta \nabla^2 \psi = 0. \tag{20.4}$$

Suppose a data-only prediction adds a small high-frequency error:

$$\widehat{\psi} = \psi + \epsilon \sin(Kx) \sin(Ky). \tag{20.5}$$

The value error is order ϵ, while the diffusion-residual contribution is order $\eta K^2 \epsilon$. Large K can therefore produce a substantial residual and current error even when value MSE is small. This miniature example captures the main scientific reason for comparing Modules 4 and 5.

20.4 A minimal numerical loss example

Suppose a batch has two standardized observations:

$$(\widetilde{\psi}_1, \widetilde{U}_1) = (0.5, -0.2), \qquad (\widetilde{\psi}_2, \widetilde{U}_2) = (-0.1, 0.4), \tag{20.6}$$

and predictions

$$(0.4, -0.1), \qquad (-0.3, 0.5). \tag{20.7}$$

Then

$$\mathrm{MSE}_\psi = \frac{(0.4 - 0.5)^2 + (-0.3 + 0.1)^2}{2} = 0.025, \tag{20.8}$$

$$\mathrm{MSE}_U = \frac{(-0.1 + 0.2)^2 + (0.5 - 0.4)^2}{2} = 0.010. \tag{20.9}$$

With equal channel weights,

$$\mathcal{L}_{\mathrm{data}} = 0.035. \tag{20.10}$$

The optimizer differentiates this scalar with respect to every weight and bias. It does not compare the prediction with a differential equation.

21 Reproducibility and implementation handoff

21.1 Configuration that must be saved

Every training run should record:

- project version and source-code commit identifier when available;
- dataset schema and case-manifest checksums;

- exact train/validation/test case identifiers;
- observed-point index lists or deterministic sampling rules;
- physical normalization and neural-scaling statistics;
- periodic feature definition and coefficient transforms;
- architecture, activation, parameter count, and initialization;
- loss definition and all weights;
- optimizer, learning-rate schedule, batch hierarchy, and update count;
- early-stopping rule and selected checkpoint;
- random seeds and software/hardware environment;
- training and validation histories; and
- final per-case metrics and prediction artifact locations.

21.2 Acceptance tests before full training

1. **Schema test:** incompatible sign conventions, normalization, field order, or coordinate shapes are rejected.
2. **Split test:** no case identifier or parameter triplet appears in more than one partition.
3. **Scaling test:** transform followed by inverse transform reproduces inputs and outputs within floating-point tolerance.
4. **Periodic-feature test:** features match at 0 and L_x or L_y.
5. **Shape test:** batches and predictions use documented row/column ordering.
6. **Loss test:** a hand-calculated batch agrees with code.
7. **Gradient test:** automatic gradients are finite and agree with finite differences for a tiny network.
8. **Overfit test:** the network can fit a very small data subset.
9. **Checkpoint test:** saving and loading reproduces identical predictions.
10. **Inference test:** full snapshots are reconstructed in the correct orientation.
11. **Derived-field test:** automatic and spectral derivatives agree on a known analytic network or Fourier field.

12. **No-physics audit:** the training graph contains only value-data and declared numerical-regularization terms.

21.3 Minimum run products

A completed baseline run should produce:

- a machine-readable frozen configuration;
- a split and observation manifest;
- preprocessing parameters;
- trained checkpoint and model metadata;
- training and validation loss histories;
- full-grid predictions for held-out cases;
- per-field and per-case metrics;
- derived physical diagnostic histories;
- timing and memory measurements; and
- a concise experiment summary linking every artifact.

21.4 Runtime quantities

Distinguish:

- T_{solver}: reference-solver time needed to generate cases;
- T_{prep}: preprocessing and data-loading time;
- T_{train}: neural training time;
- T_{point}: one point inference time at a specified batch size;
- T_{field}: complete field reconstruction time at a specified grid; and
- T_{diag}: time to compute derivatives and physical diagnostics.

Accelerator warm-up, data transfer, and batch size can dominate microbenchmarks. Report measurement protocol rather than one context-free number.

22 Common misconceptions

22.1 "A data-only neural network contains no assumptions"

False. Architecture, smooth activations, periodic features, scaling, finite capacity, optimization, and sampling all impose inductive biases. "Data-only" means no explicit governing-equation or physical-constraint term enters the training objective.

22.2 "More grid points always mean more physical information"

False. Dense points from one trajectory are correlated. Additional independent parameter cases can provide more information about the family even with fewer points per case.

22.3 "Random point splitting gives a large independent test set"

False. Points from the same trajectory share parameters, dynamics, and nearby neighbors. The apparent test set is not independent at the physical-case level.

22.4 "The network predicts the next time step"

Not in the frozen formulation. It evaluates the field directly at requested t. It is a coordinate surrogate, not an autoregressive time integrator.

22.5 "Continuous output means exact super-resolution"

False. The network is evaluable continuously, but off-grid accuracy is limited by training information, architecture, and reference resolution.

22.6 "Standardization is the same as nondimensionalization"

False. Physical nondimensionalization defines the model. Standardization is a later numerical transform fitted from training values.

22.7 "Equal raw MSE weights treat ψ and U equally"

Not if their scales differ. Separate standardization or explicit normalization is needed to interpret equal numerical weights.

22.8 "Low ψ error guarantees accurate current"

False. J depends on second spatial derivatives and can amplify high-frequency error by the squared wavenumber.

22.9 "Computing a PDE residual makes the model a PINN"

Only if the residual influences training. Evaluation-only residuals diagnose a data-only model without changing its category.

22.10 "Periodic training data automatically produce a periodic network"

Not exactly. A network may approximate endpoint agreement but still form a seam. A periodic input representation makes equality structural.

22.11 "The best test seed is the reported model"

That is selection bias. Report a predeclared selection protocol and the distribution across repeated training seeds.

22.12 "This is already an M3D-C1 surrogate"

No. It is a surrogate for an independent synthetic two-field reduced-MHD dataset in an M3D-C1-shaped future workflow. Genuine M3D-C1 data are not introduced until Module 7.

23 Module 4 computational contract

23.1 Input-output contract

$$\text{Raw inputs: } (x, y, t, \eta, \nu, A_0), \tag{23.1}$$

$$\text{Processed inputs: } \boldsymbol{z} \text{ from periodic encoding and stored training transforms}, \tag{23.2}$$

$$\text{Primary outputs: } (\widehat{\psi}, \widehat{U}), \tag{23.3}$$

$$\text{Evaluation-only derived fields: } \widehat{J} = -\nabla^2\widehat{\psi}, \quad \widehat{\omega} = \nabla^2\widehat{U}. \tag{23.4}$$

23.2 Data contract

Only verified Module 2 cases accepted by the versioned Module 3 schema may enter training. Complete case identity and provenance survive point flattening. Splits are assigned by case and

frozen before preprocessing or point selection.

23.3 Loss contract

$$\mathcal{L}_{\text{train}} = \lambda_\psi \,\text{MSE}(\widehat{\widetilde{\psi}}, \widetilde{\psi}) + \lambda_U \,\text{MSE}(\widehat{\widetilde{U}}, \widetilde{U}) + \mathcal{L}_{\text{numerical reg}}, \tag{23.5}$$

where $\mathcal{L}_{\text{numerical reg}}$ is zero by default or a declared nonphysical regularizer such as validation-selected weight decay. Physics, derivative-target, J, and ω terms are excluded from the primary baseline.

23.4 Evaluation contract

Evaluation uses complete held-out cases and reports primary fields, derived fields, time histories, physical diagnostics, residuals, periodicity, interpolation/extrapolation category, run-to-run variation, and compute cost. Metrics are computed after inverse scaling.

23.5 Fairness contract for Module 5

Module 5 inherits the same cases, observed points, preprocessing, periodic representation, architecture family, data loss, and scoring. Added collocation points, physics weights, optimization changes, and compute are documented explicitly.

24 Summary and bridge to Module 5

Module 4 establishes the strongest reasonable neural surrogate that learns only from stored field values. Its primary map is

$$(x, y, t, \eta, \nu, A_0) \longmapsto (\widehat{\psi}, \widehat{U}). \tag{24.1}$$

The spatial coordinates are represented periodically, time and parameters are transformed using stored training-only statistics, and ψ and U are standardized separately. A smooth MLP predicts both fields. Training minimizes only a case-balanced data mismatch.

Scientific validity depends at least as much on dataset design as on architecture. Complete parameter cases, not random points from every trajectory, define train, validation, and test partitions. Parameter interpolation, parameter extrapolation, temporal interpolation, and temporal extrapolation are distinct claims. Dense grid points do not replace independent physical cases.

The data-only model is evaluated physically even though it is not trained physically. Current and vorticity are obtained from second derivatives, global energies and peak current are reconstructed, periodicity is tested, and the reduced-MHD residuals are measured without contributing gradients. These diagnostics reveal whether a low value error corresponds to a physically coherent continuous field.

Module 5 begins from this exact experiment. It retains the learned map, case splits, observations, preprocessing, architecture family, and data loss, then adds

$$\mathcal{L} = w_{\text{data}}\mathcal{L}_{\text{data}} + w_{\text{PDE}}\mathcal{L}_{\text{PDE}} + w_{\text{IC}}\mathcal{L}_{\text{IC}} + w_{\text{BC}}\mathcal{L}_{\text{BC}} + w_{\text{global}}\mathcal{L}_{\text{global}}. \tag{24.2}$$

Only that controlled transition permits a defensible answer to the project question: whether explicit reduced-MHD information improves accuracy, data efficiency, generalization, or physical consistency enough to justify its additional optimization and computational cost.

A Linear-algebra and calculus foundations

A.1 Scalar, vector, and matrix

A scalar is one number. A vector is an ordered list of numbers. A matrix is a rectangular array. If $\boldsymbol{W} \in \mathbb{R}^{m\times n}$ and $\boldsymbol{h} \in \mathbb{R}^n$, then $\boldsymbol{W}\boldsymbol{h} \in \mathbb{R}^m$. The entry in output position i is

$$(\boldsymbol{W}\boldsymbol{h})_i = \sum_{j=1}^{n} W_{ij} h_j. \tag{A.1}$$

An affine map adds a bias:

$$\boldsymbol{a} = \boldsymbol{W}\boldsymbol{h} + \boldsymbol{b}. \tag{A.2}$$

A.2 Gradient and Jacobian

For scalar $f(\boldsymbol{x})$, the gradient is the vector of partial derivatives. For vector function $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian is the $m \times n$ matrix

$$J_{ij} = \frac{\partial f_i}{\partial x_j}. \tag{A.3}$$

Automatic differentiation computes products involving these derivatives without necessarily forming every Jacobian entry explicitly.

A.3 Chain rule through scaling

If $\widetilde{x} = (x - m_x)/s_x$ and $\widehat{\psi} = m_\psi + s_\psi \widehat{\widetilde{\psi}}(\widetilde{x})$, then

$$\frac{\partial \widehat{\psi}}{\partial x} = \frac{s_\psi}{s_x} \frac{\partial \widehat{\widetilde{\psi}}}{\partial \widetilde{x}}, \tag{A.4}$$

$$\frac{\partial^2 \widehat{\psi}}{\partial x^2} = \frac{s_\psi}{s_x^2} \frac{\partial^2 \widehat{\widetilde{\psi}}}{\partial \widetilde{x}^2}. \tag{A.5}$$

With periodic encoding, another chain-rule layer enters. For $c_x = \cos(2\pi x/L_x)$ and $s_x = \sin(2\pi x/L_x)$,

$$\frac{\partial \widehat{\psi}}{\partial x} = \frac{2\pi}{L_x} \left(c_x \frac{\partial \widehat{\psi}}{\partial s_x} - s_x \frac{\partial \widehat{\psi}}{\partial c_x} \right). \tag{A.6}$$

Automatic differentiation can propagate these factors if the feature construction is inside the differentiable graph. Hand scaling outside that graph requires explicit care.

A.4 Mean and standard deviation

For N training values a_n, the population-form mean and standard deviation used for deterministic preprocessing may be written

$$m_a = \frac{1}{N}\sum_{n=1}^{N} a_n, \tag{A.7}$$

$$s_a = \left[\frac{1}{N}\sum_{n=1}^{N}(a_n - m_a)^2\right]^{1/2}. \tag{A.8}$$

The code must freeze whether the divisor is N or $N-1$. Either can be made consistent; mixing conventions prevents exact reproduction.

B Metric catalog

For a reference vector $\boldsymbol{a}$ and prediction $\widehat{\boldsymbol{a}}$ with N entries:

B.1 Mean absolute error

$$\mathrm{MAE} = \frac{1}{N}\sum_{n=1}^{N} |\widehat{a}_n - a_n|. \tag{B.1}$$

B.2 Mean squared error and root mean squared error

$$\mathrm{MSE} = \frac{1}{N}\sum_{n=1}^{N} (\widehat{a}_n - a_n)^2, \tag{B.2}$$

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}. \tag{B.3}$$

B.3 Relative L_2 error

$$\epsilon_{2,a} = \frac{\left[\sum_n (\widehat{a}_n - a_n)^2\right]^{1/2}}{\left[\sum_n a_n^2\right]^{1/2} + \epsilon_{\mathrm{floor}}}. \tag{B.4}$$

B.4 Relative maximum error

$$\epsilon_{\infty,a} = \frac{\max_n |\widehat{a}_n - a_n|}{\max_n |a_n| + \epsilon_{\mathrm{floor}}}. \tag{B.5}$$

B.5 Coefficient of determination

The coefficient

$$R^2 = 1 - \frac{\sum_n (\widehat{a}_n - a_n)^2}{\sum_n (a_n - \overline{a})^2} \tag{B.6}$$

compares squared error with deviation from the sample mean. R^2 can be negative on held-out data and can be misleading for nearly constant fields. It is supplementary, not a replacement for physically interpretable errors.

B.6 Peak and integral errors

For scalar diagnostic $d(t)$,

$$\epsilon_d(t) = \frac{|\widehat{d}(t) - d(t)|}{|d(t)| + \epsilon_{\text{floor}}}. \tag{B.7}$$

For a history, report time-resolved error and an integrated measure such as

$$\epsilon_{d,\text{hist}} = \frac{\left[\sum_k (\widehat{d}_k - d_k)^2\right]^{1/2}}{\left[\sum_k d_k^2\right]^{1/2} + \epsilon_{\text{floor}}}. \tag{B.8}$$

C Symbol table

Symbol	Meaning	Status or units
x, y	Cartesian coordinates in the periodic plane	dimensionless
t	Time	dimensionless
L_x, L_y	Period lengths of the spatial domain	dimensionless
η	Magnetic diffusivity parameter	dimensionless
ν	Kinematic viscosity parameter	dimensionless
A_0	Initial perturbation amplitude parameter	dimensionless
$\boldsymbol{\mu}$	Case parameter vector (η, ν, A_0)	dimensionless
ψ	Magnetic-flux function	dimensionless
U	Flow stream function	dimensionless
J	Current variable $-\nabla^2\psi$	dimensionless
ω	Vorticity $\nabla^2 U$	dimensionless
$\boldsymbol{s}$	Raw network input vector	mixed dimensionless entries
$\boldsymbol{z}$	Processed network input vector	numerically scaled
$\boldsymbol{q}$	Target output vector (ψ, U)	dimensionless
$\widehat{a}$	Neural prediction of quantity a	same status as a
$\widetilde{a}$	Numerically transformed version of a	scaled
m_a, s_a	Stored training-set location and scale for a	same units as a

Symbol	Meaning	Status or units
$\mathcal{N}_{\boldsymbol{\theta}}$	Neural-network function	numerical map
$\boldsymbol{\theta}$	Vector of all trainable weights and biases	numerical parameters
$\boldsymbol{W}^{(\ell)}$	Weight matrix at layer ℓ	trainable
$\boldsymbol{b}^{(\ell)}$	Bias vector at layer ℓ	trainable
$\boldsymbol{a}^{(\ell)}$	Pre-activation at layer ℓ	numerical
$\boldsymbol{h}^{(\ell)}$	Hidden representation at layer ℓ	numerical
σ	Activation function	nonlinear function
L	Number of hidden layers	integer
n_ℓ	Width of layer ℓ	integer
N_θ	Number of trainable scalar parameters	integer
$\mathcal{D}_{\mathrm{tr}}$	Training observations	dataset subset
$\mathcal{C}_{\mathrm{tr}}$	Training case identifiers	case subset
$\mathcal{C}_{\mathrm{va}}$	Validation case identifiers	case subset
$\mathcal{C}_{\mathrm{te}}$	Test case identifiers	case subset
$\mathcal{B}$	Mini-batch of supervised examples	dataset subset
$\mathcal{L}_{\mathrm{data}}$	Data mismatch loss	scalar
λ_ψ, λ_U	Output-channel data-loss weights	dimensionless
α	Learning rate or declared regularization coefficient, according to context	dimensionless numerical choice
β_1, β_2	Adam exponential-moment coefficients	dimensionless
$\boldsymbol{m}_k, \boldsymbol{v}_k$	Adam first- and second-moment estimates	optimizer state
e_a	Prediction error $\widehat{a} - a$	same status as a
$\epsilon_{\mathrm{floor}}$	Declared small denominator floor in relative metrics	same status as denominator
E_B, E_K	Magnetic and kinetic energies	dimensionless integrals
$\mathcal{R}_\psi, \mathcal{R}_\omega$	Reduced-MHD PDE residuals	evaluation-only in Module 4
T_{solver}	Reference-case generation time	time
T_{train}	Neural training time	time
T_{field}	Full-field inference time	time

D Acronym and terminology table

Term	Meaning
AD	Automatic differentiation; exact chain-rule differentiation of a computational graph up to floating-point effects
BC	Boundary condition
CPU	Central processing unit

Term	Meaning
Data leakage	Use of validation or test information in training, preprocessing, or model selection
Epoch	Approximately one pass through a fixed training set; must be clarified for resampled data
Feature	Numerical input supplied to a learning model
FNO	Fourier neural operator
Fourier feature	Sine-cosine transformation used to represent frequency content or periodicity
GPU	Graphics processing unit
HDF5	Hierarchical Data Format version 5, a scientific array container
Hyperparameter	Design choice not updated by ordinary backpropagation, such as width or learning rate
IC	Initial condition
Inductive bias	Assumption or preference introduced by representation, architecture, loss, or optimization
Inference	Evaluation of a trained model
Interpolation	Prediction within the support of represented inputs
Extrapolation	Prediction outside the support of represented inputs
Label or target	Desired output paired with a supervised input
MAE	Mean absolute error
MHD	Magnetohydrodynamics
MLP	Multilayer perceptron, a feedforward network of affine layers and nonlinear activations
MSE	Mean squared error
Neural operator	Learned mapping between function spaces
Optimizer	Algorithm that updates trainable parameters using loss derivatives
Overfitting	Low training error with poor performance on unseen data
PDE	Partial differential equation
PINN	Physics-informed neural network
RMHD	Reduced magnetohydrodynamics; the precise reduction must be stated
RMSE	Root mean squared error
Seed	Integer used to initialize a pseudorandom sequence
Snapshot	Complete spatial field at one stored time
Spectral bias	Tendency of a coordinate network and its optimization to fit low frequencies before high frequencies
Standardization	Numerical centering and scaling using stored statistics
Supervised learning	Learning from paired inputs and desired outputs

Term	Meaning
Surrogate	Computational approximation to another model's input-output behavior
Test set	Untouched data used for final performance estimation after design freeze
Training set	Data used to update model parameters
Underfitting	Failure to represent or optimize even the training relationship adequately
Validation set	Held-out data used for model and hyperparameter selection
Weight decay	Numerical regularization that shrinks trainable parameters

E Concept questions and answer sketches

Questions

Q1. Why is Module 4 necessary if the project's main interest is a PINN?

Q2. What makes the Module 4 model parameterized?

Q3. What is the difference between one simulation case and one supervised point example?

Q4. Why is a random point split across every trajectory scientifically weak?

Q5. Why must scaling statistics be computed from training cases only?

Q6. What is the difference between physical nondimensionalization and neural standardization?

Q7. Why use both sine and cosine for a periodic coordinate?

Q8. Why is tanh a more transferable activation than ReLU for the Module 4 to Module 5 comparison?

Q9. What do network weights and biases represent?

Q10. Why does adding many linear layers without activations not create a nonlinear model?

Q11. Why is equal raw MSE not necessarily equal treatment of ψ and U?

Q12. What is backpropagation, and how does it differ from an optimizer?

Q13. What is the difference between an epoch and an optimizer update?

Q14. Why can J be inaccurate when ψ looks accurate?

Q15. Can a data-only model be evaluated with a PDE residual without becoming a PINN?

Q16. Why is time interpolation not forecasting?

Q17. What is the difference between a coordinate network and a neural operator?

Q18. Which quantities should be held fixed in the primary Module 4 versus Module 5 comparison?

Q19. Why should several random seeds be reported?

Q20. What claim about M3D-C1 is justified after completing Module 4?

Answer sketches

A1. It measures what can be learned from data alone, so changes caused by added physics information can be identified rather than assumed.

A2. The case parameters (η, ν, A_0) are network inputs, allowing one set of weights to represent multiple cases.

A3. A case is a complete trajectory at fixed physical parameters. A point example is one coordinate-time-parameter input paired with one field value pair.

A4. Training and test points share the same parameters and trajectory and may be neighbors, so the test measures within-case interpolation rather than unseen-case generalization.

A5. Validation/test statistics leak information about held-out distributions and make performance estimates optimistic or deployment assumptions unrealistic.

A6. Nondimensionalization defines the physics relative to reference scales; standardization is a later invertible numerical transform used for optimization.

A7. Together they map a circle without a discontinuity and distinguish phase; either sine or cosine alone maps multiple positions to the same value.

A8. Tanh is smooth through the derivative orders needed by the strong-form PDE residual; ReLU has unsuitable classical second derivatives.

A9. They are trainable numerical coefficients controlling affine combinations and offsets of internal features.

A10. A composition of affine maps is still affine. Nonlinear activations are required to represent nonlinear dependence.

A11. A larger-variance output contributes larger squared numerical errors and can dominate unless channels are separately scaled or weighted.

A12. Backpropagation computes loss derivatives by the chain rule. An optimizer uses those derivatives to choose parameter updates.

A13. An update changes parameters once; an epoch is approximately one pass through a defined finite training set and may be ambiguous with resampling.

A14. $J = -\nabla^2\psi$ depends on second derivatives, which amplify high-frequency errors by squared wavenumber.

A15. Yes. It remains data-only if the residual is diagnostic and never contributes a gradient or model-selection rule that is falsely described as training-free.

A16. If the training data span the tested time interval, unused times inside it are interpolation. Forecasting requires evaluation beyond the trained interval.

A17. A coordinate network maps finite coordinates and parameters to field values. A neural operator maps an input function to an output function.

A18. Case splits, labeled observations, preprocessing, coordinate representation, architecture capacity, data loss, seeds, and evaluation should be matched; additional PINN collocation and compute are reported.

A19. Optimization is stochastic and seed-sensitive. A distribution distinguishes reliable improvement from a favorable initialization.

A20. Only that a data-only surrogate has been tested on synthetic two-field reduced-MHD cases in an M3D-C1-shaped workflow. No validated actual M3D-C1 surrogate claim follows.

References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press (2016), https://www.deeplearningbook.org/.

[2] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[3] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems* **2**, 303–314 (1989), https://doi.org/10.1007/BF02551274.

[4] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, PMLR **9**, 249–256 (2010), https://proceedings.mlr.press/v9/glorot10a.html.

[5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *Third International Conference on Learning Representations* (2015), https://arxiv.org/abs/1412.6980.

[6] M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, and R. Ng, "Fourier features let networks learn high frequency functions in low dimensional domains," *Advances in Neural Information Processing Systems* **33**, 7537–7547 (2020), https://proceedings.neurips.cc/paper/2020/hash/55053683268957697aa39fba6f231c68-Abstract.html.

[7] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," *Nature Machine Intelligence* **3**, 218–229 (2021), https://doi.org/10.1038/s42256-021-00302-5.

[8] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," *Ninth International Conference on Learning Representations* (2021), https://openreview.net/forum?id=c8P9NQVtmnO.

[9] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics* **378**, 686–707 (2019), https://doi.org/10.1016/j.jcp.2018.10.045.

[10] J. P. Goedbloed and S. Poedts, *Principles of Magnetohydrodynamics: With Applications to Laboratory and Astrophysical Plasmas*, Cambridge University Press (2004).